

Ngugi Sandra Lucy Wahu

Supervisor: Clive Onsomu

12/10/2025

System Design Document

Minion Me Service Marketplace

18/03030

FINAL YEAR 1 BISF 3500

Table of Contents

1. Introduction	5
1.2 Scope	5
1.3 Intended Audience	6
2. System Overview	7
Minion Me is a web-based service marketplace platform designed to connect clients with vetted local service providers (“Minions”). The system enables task posting, provider matching, secure payments, task tracking, in-app notifications, and performance rating.	
.....	Error! Bookmark not defined.
The system adopts a three-tier architecture:.....	7
• Presentation Layer (Frontend)	7
• Application Layer (Backend API)	7
• Data Layer (Database).....	7
External services include Mobile Money APIs for payments.....	7
3. System Architecture.....	7
The system follows a client–server architecture with RESTful APIs.	
Architecture Characteristics:	7
• Modular and scalable.....	7
• Separation of concerns	7
• Secure communication over HTTPS.....	7
• Mobile-first design approach	7
3.2 High-Level Architecture Components.....	7
• Web Application (React)	7
• Backend API (Node.js).....	7
• MySQL Database	7
• Mobile Money API	7
• In-App Notification Module	7
4. Component Design	7
4.1 Presentation Layer (Frontend).....	7
Technology: React.js	7
Responsibilities:	7
• User registration and login	8
• Task creation and browsing	8

• Provider profile viewing	8
• Task progress updates	8
• Display of in-app notifications	8
• Payment initiation interface	8
• Key UI Components:.....	8
• Login & Registration Pages	8
• Client Dashboard	8
• Service Provider Dashboard.....	8
• Task Posting Form	8
• Task Tracking View	8
• Notifications Panel.....	8
• Ratings & Reviews Interface.....	8
4.2 Application Layer (Backend API)	9
Technology: Node.js.....	9
Responsibilities:	9
• Business logic execution	9
• Authentication and authorization.....	9
• Task lifecycle management.....	9
• Payment processing coordination	9
• Notification generation.....	9
• Data validation and error handling	9
• Core Backend Modules:	9
• Authentication Module	9
• User Management Module.....	9
• Task Management Module	9
• Payment Module.....	9
• Notification Module.....	9
• Rating & Review Module.....	9
• Admin Management Module	9
4.3 Data Layer (Database)	9
Technology: MySQL	9
Responsibilities:	9

• Persistent data storage	9
• Data integrity enforcement.....	9
• Transaction logging.....	9
5. Database Design	9
5.1 Entity Relationship Overview.....	10
Key Entities:	10
• User	10
• ServiceProvider	10
• Task.....	10
• Bid.....	10
• Payment	10
• Notification	10
• Review	10
5.2 Database Tables (Summary).....	10
Users Table	10
• user_id (PK)	10
• full_name.....	10
• email	10
• phone_number	10
• password_hash.....	10
• role (client/provider/admin)	10
• verification_status.....	10
• created_at	10
Service_Providers Table.....	10
• provider_id (PK)	10
• user_id (FK)	10
• skills.....	10
• rating	10
• verification_documents	10
Tasks Table.....	10
• task_id (PK).....	11
• client_id (FK)	11

• provider_id (FK)	11
• description	11
• location	11
• budget	11
• status	11
• created_at	11
Payments Table	11
• payment_id (PK)	11
• task_id (FK)	11
• amount	11
• payment_status	11
• transaction_reference	11
• created_at	11
• Notifications Table	11
• notification_id (PK)	11
• user_id (FK)	11
• message	11
• status (read/unread)	11
• created_at	11
Reviews Table	11
• review_id (PK)	11
• task_id (FK)	11
• rating	11
• comment	12
6. System Workflow Design	12
6.1 User Registration Workflow	12
6.2 Task Posting and Assignment Workflow	12
6.3 Payment Workflow	12
6.4 Notification Workflow	12
7. Interface Design	13
7.1 User Interface Design	13
7.2 External Interface Design	13

8. Security Design	14
8.1 Authentication & Authorization	14
• Password hashing using BCrypt	14
• Role-based access control (RBAC)	14
• Session management using JWT tokens	14
8.2 Data Security	14
• HTTPS for all communications	14
• Encrypted payment transactions	14
• Secure storage of sensitive data.....	14
8.3 Application Security	14
• Input validation.....	14
• Protection against SQL Injection and XSS	14
• Account lockout after multiple failed login attempts.....	14
9. Deployment Design	14
9.1 Deployment Environment.....	14
9.2 Deployment Architecture.....	14
10. System Maintenance and Scalability	15
10.1 Maintenance.....	15
10.2 Scalability.....	15
11. Conclusion	15

1. Introduction

This System Design Document (SDD) describes the architecture and detailed design of the Minion Me Service Marketplace system. It translates the requirements defined in the Software Requirements Specification (SRS) into a detailed technical design that guides system development and implementation.

1.2 Scope

This document covers:

- System architecture
- Component design
- Database design

- Interface design
- Security design
- Deployment environment

The document is intended to be used by developers, system testers, supervisors, and future maintainers of the system.

1.3 Intended Audience

- Software Developers
- Project Supervisor and Examiners
- System Testers
- Technical Documentation Reviewers

2. System Overview

Minion Me is a web-based service marketplace that connects clients with vetted local service providers (“Minions”). The system enables task posting, provider matching, secure payments, task tracking, in-app notifications, and performance ratings.

The system adopts a three-tier architecture:

- Presentation Layer (Frontend)
- Application Layer (Backend API)
- Data Layer (Database)

External services include Mobile Money APIs for payments.

3. System Architecture

The system follows a client–server architecture with RESTful APIs.

Architecture Characteristics:

- Modular and scalable
- Separation of concerns
- Secure communication over HTTPS
- Mobile-first design approach

3.2 High-Level Architecture Components

- Web Application (React)
- Backend API (Node.js)
- MySQL Database
- Mobile Money API
- In-App Notification Module

4. Component Design

4.1 Presentation Layer (Frontend)

Technology: React.js

Responsibilities:

- User registration and login
- Task creation and browsing
- Provider profile viewing
- Task progress updates
- Display of in-app notifications
- Payment initiation interface
- Key UI Components:
- Login & Registration Pages
- Client Dashboard
- Service Provider Dashboard
- Task Posting Form
- Task Tracking View
- Notifications Panel
- Ratings & Reviews Interface

4.2 Application Layer (Backend API)

Technology: Node.js

Responsibilities:

- Business logic execution
- Authentication and authorization
- Task lifecycle management
- Payment processing coordination
- Notification generation
- Data validation and error handling
- Core Backend Modules:
 - Authentication Module
 - User Management Module
 - Task Management Module
 - Payment Module
 - Notification Module
 - Rating & Review Module
 - Admin Management Module

4.3 Data Layer (Database)

Technology: MySQL

Responsibilities:

- Persistent data storage
- Data integrity enforcement
- Transaction logging

5. Database Design

5.1 Entity Relationship Overview

Key Entities:

- User
- ServiceProvider
- Task
- Bid
- Payment
- Notification
- Review

5.2 Database Tables (Summary)

Users Table

- user_id (PK)
- full_name
- email
- phone_number
- password_hash
- role (client/provider/admin)
- verification_status
- created_at

Service_Providers Table

- provider_id (PK)
- user_id (FK)
- skills
- rating
- verification_documents

Tasks Table

- task_id (PK)
- client_id (FK)
- provider_id (FK)
- description
- location
- budget
- status
- created_at

Payments Table

- payment_id (PK)
- task_id (FK)
- amount
- payment_status
- transaction_reference
- created_at
- Notifications Table
- notification_id (PK)
- user_id (FK)
- message
- status (read/unread)
- created_at

Reviews Table

- review_id (PK)
- task_id (FK)
- rating

- comment

6. System Workflow Design

6.1 User Registration Workflow

1. User submits registration details
2. System validates input
3. Account created
4. User verification process initiated
5. User redirected to dashboard

6.2 Task Posting and Assignment Workflow

1. Client posts task
2. Providers view task listings
3. Providers submit bids or accept fixed price
4. Client selects provider
5. Task status updated to “Assigned”
6. In-app notifications sent to both parties

6.3 Payment Workflow

1. Client initiates payment
2. Backend communicates with Mobile Money API
3. Payment confirmation received
4. Task marked as “Paid”
5. Transaction recorded

6.4 Notification Workflow

1. System event occurs (task update, payment, assignment)
2. Notification generated in backend
3. Notification stored in database
4. Notification displayed in user dashboard (in-app)

7. Interface Design

7.1 User Interface Design

- Responsive layout
- Simple navigation
- Clear call-to-action buttons
- Minimal text input
- Icons for notifications and status indicators

7.2 External Interface Design

- Mobile Money API integration using REST
- Secure API tokens
- JSON data exchange format

8. Security Design

8.1 Authentication & Authorization

- Password hashing using BCrypt
- Role-based access control (RBAC)
- Session management using JWT tokens

8.2 Data Security

- HTTPS for all communications
- Encrypted payment transactions
- Secure storage of sensitive data

8.3 Application Security

- Input validation
- Protection against SQL Injection and XSS
- Account lockout after multiple failed login attempts

9. Deployment Design

9.1 Deployment Environment

- Cloud-hosted Linux server
- Node.js runtime
- MySQL database server
- Reverse proxy

9.2 Deployment Architecture

- Frontend hosted on web server
- Backend API hosted on application server
- Database hosted separately
- Environment variables for secrets

10. System Maintenance and Scalability

10.1 Maintenance

- Regular database backups
- Log monitoring
- Bug fixing and patching

10.2 Scalability

- Modular codebase
- Horizontal scaling of backend
- Load balancing support

11. Conclusion

This System Design Document provides a complete technical blueprint for implementing the Minion Me Service Marketplace. The design ensures scalability, security, usability, and alignment with the project's objectives of formalizing Kenya's informal service economy through a secure digital platform.